Computing on a Full Memory

Harry Buhrman	University of Amsterdam & CWI
Richard Cleve	University of Waterloo
Michal Koucký	Charles University, Prague
Bruno Loff	University of Porto & INESC-Tec
Florian Speelman	University of Copenhagen



A challenge

- Suppose you are doing a computation that requires more memory than the free memory you have available.



A challenge

- Suppose you are doing a computation that requires more memory than the free memory you have available.



- Compress $\rightarrow$ use $\rightarrow$ decompress?

A challenge

- Suppose you are doing a computation that requires more memory than the free memory you have available.



- Compress $\rightarrow$ use $\rightarrow$ decompress?
- Contents incompressible $\implies$ full memory is useless?

A challenge

- Suppose you are doing a computation that requires more memory than the free memory you have available.



- Compress $\rightarrow$ use $\rightarrow$ decompress?
- Contents incompressible $\implies$ full memory is useless?
- **Challenge:** Prove that the extra full memory is useless!

Catalytic Space

A model of computation where the challenge above is made precise

- L is logspace.

Catalytic Space

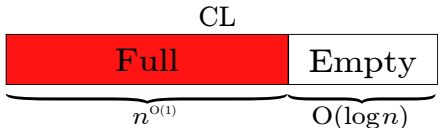
A model of computation where the challenge above is made precise

- L is logspace.
- CL is logspace + poly full-memory.

Catalytic Space

A model of computation where the challenge above is made precise

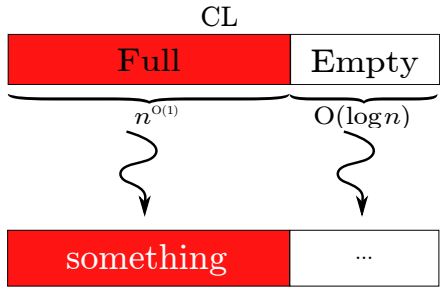
- L is logspace.
- CL is logspace + poly full-memory.



Catalytic Space

A model of computation where the challenge above is made precise

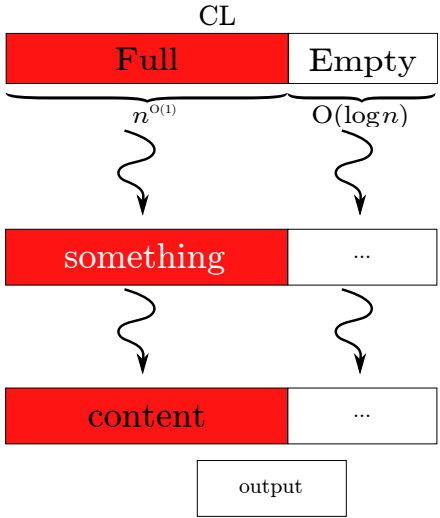
- L is logspace.
- CL is logspace + poly full-memory.



Catalytic Space

A model of computation where the challenge above is made precise

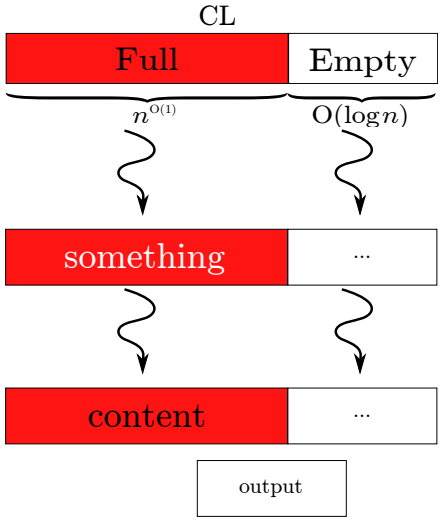
- L is logspace.
- CL is logspace + poly full-memory.



Catalytic Space

A model of computation where the challenge above is made precise

- L is logspace.
- CL is logspace + poly full-memory.
- **Question:** Is L = CL?



Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

- Sheer intuition!

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is L = CL?

Why one might lean on "YES!"

- Sheer intuition!
- At any step of the computation, the mutual information between the input and the tape contents is still $O(\log n)$.

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

- Sheer intuition!
- At any step of the computation, the mutual information between the input and the tape contents is still $O(\log n)$.
- So, in some sense, the algorithm only "remembers" $O(\log n)$ bits of the input.

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

- Sheer intuition!
- At any step of the computation, the mutual information between the input and the tape contents is still $O(\log n)$.
- So, in some sense, the algorithm only "remembers" $O(\log n)$ bits of the input.
- So maybe we can show $\mathsf{CL} = \mathsf{L}$ by maintaining a compressed representation?

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

- Sheer intuition!
- At any step of the computation, the mutual information between the input and the tape contents is still $O(\log n)$.
- So, in some sense, the algorithm only "remembers" $O(\log n)$ bits of the input.
- So maybe we can show $\mathsf{CL} = \mathsf{L}$ by maintaining a compressed representation?

Why one might lean on "NO!"

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

- Sheer intuition!
- At any step of the computation, the mutual information between the input and the tape contents is still $O(\log n)$.
- So, in some sense, the algorithm only "remembers" $O(\log n)$ bits of the input.
- So maybe we can show $\mathsf{CL} = \mathsf{L}$ by maintaining a compressed representation?

Why one might lean on "NO!"

- Either we may compress and use the freed-up space, or we have something incompressible and we can maybe use it to derandomize.

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Why one might lean on "YES!"

- Sheer intuition!
- At any step of the computation, the mutual information between the input and the tape contents is still $O(\log n)$.
- So, in some sense, the algorithm only "remembers" $O(\log n)$ bits of the input.
- So maybe we can show $\mathsf{CL} = \mathsf{L}$ by maintaining a compressed representation?

Why one might lean on "NO!"

- Either we may compress and use the freed-up space, or we have something incompressible and we can maybe use it to derandomize.
- Barrington's Theorem shows NC_1 can be computed with $O(1)$ bits of memory. This is also unexpected and non-trivial, so maybe we can use similar trickery here.

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

We used to think YES! but now we think NO!

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $L = CL$?

We used to think YES! but now we think NO!

Theorem

$RL \subseteq CL$

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $L = CL$?

We used to think YES! but now we think NO!

Theorem
$RL \subseteq CL$ (compress-or-derandomize)

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

We used to think YES! but now we think NO!

Theorem
$\mathsf{RL} \subseteq \mathsf{CL}$ (compress-or-derandomize)

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{CL}$

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

We used to think YES! but now we think NO!

Theorem
$\mathsf{RL} \subseteq \mathsf{CL}$ (compress-or-derandomize)

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{CL}$

(Barrington-like argument)

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

We used to think YES! but now we think NO!

Theorem
$\mathsf{RL} \subseteq \mathsf{CL}$

(compress-or-derandomize)

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{CL}$

(Barrington-like argument)

Fact
$\mathsf{NL}, \#\mathsf{L}, \mathsf{RL}, \mathsf{LOGCFL}, \mathsf{VP} \subseteq \mathsf{TC}^1$

Catalytic Space

A model of computation where the challenge above is made precise

Question: Is $\mathsf{L} = \mathsf{CL}$?

We used to think YES! but now we think NO!

Theorem
$\mathsf{RL} \subseteq \mathsf{CL}$

(compress-or-derandomize)

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{CL}$

(Barrington-like argument)

Fact
$\mathsf{NL}, \#\mathsf{L}, \mathsf{RL}, \mathsf{LOGCFL}, \mathsf{VP} \subseteq \mathsf{TC}^1$

Belief
$\mathsf{TC}^1 \not\subseteq \mathsf{L}$, *hence* $\mathsf{CL} \not\subseteq \mathsf{L}$

Today

A full proof that $\#\mathsf{L} \subseteq \mathsf{CL}$.

L vs. #L, or Connectivity vs. Counting Walks

L

The class of functions computed by deterministic TMs allowed to use $O(\log n)$ cells of working memory.

L vs. #L, or Connectivity vs. Counting Walks

L

The class of functions computed by deterministic TMs allowed to use $O(\log n)$ cells of working memory.

#L

Computed by *counting* the number of accepting paths in non-deterministic TMs using $O(\log n)$ work memory.

L vs. #L, or Connectivity vs. Counting Walks

L

The class of functions computed by deterministic TMs allowed to use $O(\log n)$ cells of working memory.

#L

Computed by *counting* the number of accepting paths in non-deterministic TMs using $O(\log n)$ work memory.

$\Uparrow$ Complete $\Uparrow$

Connectivity

Given an undirected graph G with n vertices, and two vertices s and t of G: is there a path between s and t?

L vs. #L, or Connectivity vs. Counting Walks

L

The class of functions computed by deterministic TMs allowed to use $O(\log n)$ cells of working memory.

#L

Computed by *counting* the number of accepting paths in non-deterministic TMs using $O(\log n)$ work memory.

⇑ Complete ⇑

⇑ Complete ⇑

Connectivity

Given an undirected graph G with n vertices, and two vertices s and t of G: is there a path between s and t?

Counting Walks

Given G with n vertices, and two vertices s, t of G: how many *walks* of length n exist between s and t?

A simple polynomial-time algorithm for counting walks

Counting Walks

Given an undirected graph G with n vertices, and two vertices s and t of G, how many *walks* of length n exist between s and t?

A simple polynomial-time algorithm for counting walks

Counting Walks

Given an undirected graph G with n vertices, and two vertices s and t of G, how many *walks* of length n exist between s and t?

Simple Claim

Let $M \in \mathbb{Z}^{n \times n}$ be G's adjacency matrix. Then

$$(M^n)_{st},$$

is exactly the number of length-n walks between s and t.

A simple polynomial-time algorithm for counting walks

Counting Walks

Given an undirected graph G with n vertices, and two vertices s and t of G, how many *walks* of length n exist between s and t?

Simple Claim $(*)$

Let $M \in \mathbb{Z}^{n \times n} \mod 2^{n^2}$ be G's adjacency matrix. Then

$$(M^n)_{st},$$

is exactly the number of length-n walks between s and t.

A simple polynomial-time algorithm for counting walks

Counting Walks

Given an undirected graph G with n vertices, and two vertices s and t of G, how many *walks* of length n exist between s and t?

Simple Claim $(*)$

Let $M \in \mathbb{Z}^{n \times n} \mod 2^{n^2}$ be G's adjacency matrix. Then

$$(M^n)_{st},$$

is exactly the number of length-n walks between s and t.

Corollary

If CL *can compute* M^n *over the ring* $n \times n$ *matrices with* n^2*-bit entries, then* $\#\mathsf{L} \subseteq \mathsf{CL}$.

A simple polynomial-time algorithm for counting walks

Counting Walks

Given an undirected graph G with n vertices, and two vertices s and t of G, how many *walks* of length n exist between s and t?

Simple Claim $(*)$

Let $M \in \mathbb{Z}^{n \times n} \mod 2^{n^2}$ be G's adjacency matrix. Then

$$(M^n)_{st},$$

is exactly the number of length-n walks between s and t.

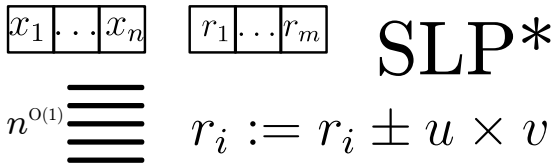
Corollary

If CL *can compute* M^n *over the ring* $n \times n$ *matrices with* n^2*-bit entries, then* $\#\mathsf{L} \subseteq \mathsf{CL}$.

THIS IS WHAT WE WILL NOW SHOW.

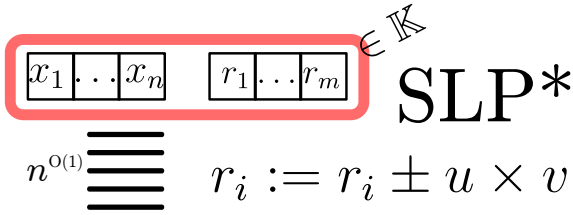
Transparent Straight-Line Programs

An algebraic model using "full memory"



Transparent Straight-Line Programs

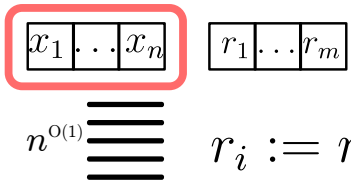
An algebraic model using "full memory"



Registers over some ring $\mathbb{K}$

Transparent Straight-Line Programs

An algebraic model using “full memory”



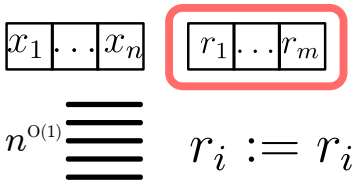
$$\mathrm{SLP}^*$$

$$r_i := r_i \pm u \times v$$

Read-only input registers

Transparent Straight-Line Programs

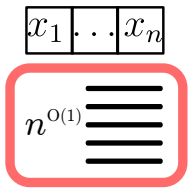
An algebraic model using "full memory"

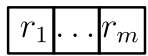


Read-write work registers

Transparent Straight-Line Programs

An algebraic model using "full memory"





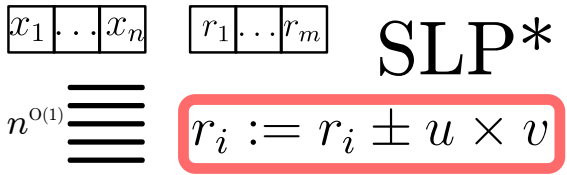
SLP^*

$r_i := r_i \pm u \times v$

A *program* is a sequence of instructions

Transparent Straight-Line Programs

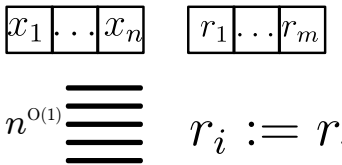
An algebraic model using "full memory"



u, v constants or registers $\neq r_i$

Transparent Straight-Line Programs

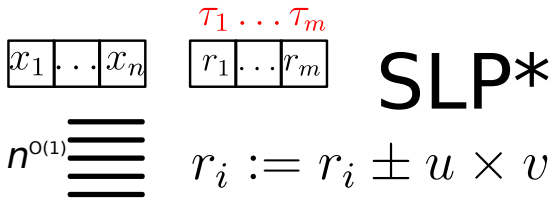
An algebraic model using "full memory"



In order to compute $f : \mathbb{K}^n \to \mathbb{K}$

Transparent Straight-Line Programs

An algebraic model using "full memory"



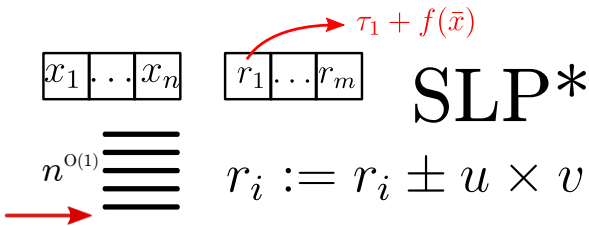
In order to compute $f : \mathbb{K}^n \to \mathbb{K}$

At the **start**:

$r_i = \tau_i$

Transparent Straight-Line Programs

An algebraic model using "full memory"



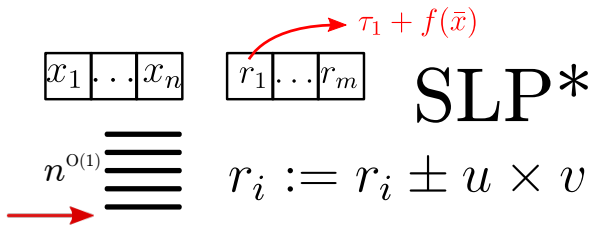
In order to compute $f : \mathbb{K}^n \to \mathbb{K}$

At the **start**:
$r_i = \tau_i$

At the **end**:
$r_1 = \tau_1 + f(x_1, \ldots, x_n)$

Transparent Straight-Line Programs

An algebraic model using "full memory"



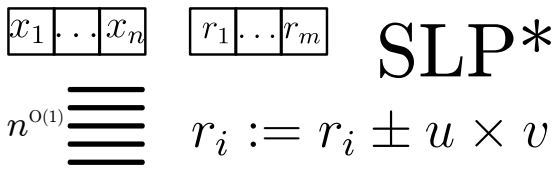
In order to compute $f : \mathbb{K}^n \to \mathbb{K}$

At the **start**:
$r_i = \tau_i$

At the **end**:
$r_1 = \tau_1 + f(x_1, \ldots, x_n)$

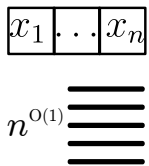
(This is why the programs are called "Transparent")

Properties of SLP*



Every program is reversible:

Properties of SLP*





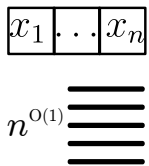
SLP*

$$r_i := r_i \pm u \times v$$

Every program is reversible:

$$r_1 := r_1 + u \times v$$
$$r_2 := r_2 - u' \times v'$$
$$r_3 := r_3 + u'' \times v''$$

Properties of SLP*



$r_1 \ldots r_m$

SLP*

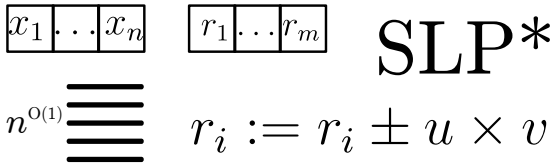
$r_i := r_i \pm u \times v$

Every program is reversible:

$$r_1 := r_1 + u \times v$$
$$r_2 := r_2 - u' \times v'$$
$$r_3 := r_3 + u'' \times v''$$

$$r_3 := r_3 - u'' \times v''$$
$$r_2 := r_2 + u' \times v'$$
$$r_1 := r_1 - u \times v$$

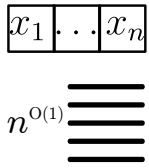
Properties of SLP*



Theorem

If $\mathbb{K}$*-product,* $\mathbb{K}$*-addition and* $\mathbb{K}$*-subtraction are in* L*, then* SLP* $\subseteq$ CL

Properties of SLP*



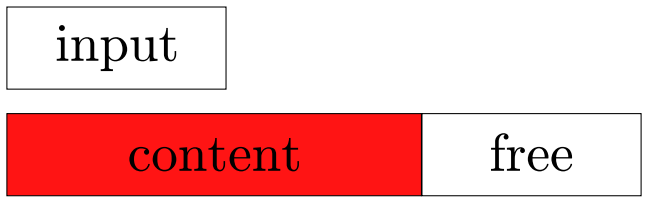


$$\text{SLP*}$$

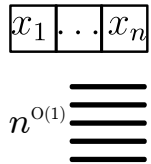
$$r_i := r_i \pm u \times v$$

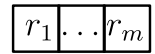
Theorem

If $\mathbb{K}$*-product,* $\mathbb{K}$*-addition and* $\mathbb{K}$*-subtraction are in* L*, then* SLP* $\subseteq$ CL



Properties of SLP*



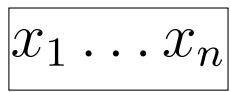


$$\text{SLP*}$$

$$r_i := r_i \pm u \times v$$

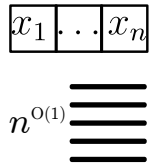
Theorem

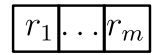
If $\mathbb{K}$*-product,* $\mathbb{K}$*-addition and* $\mathbb{K}$*-subtraction are in* L*, then* SLP* $\subseteq$ CL





Properties of SLP*





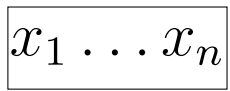
$$\text{SLP*}$$

$$r_i := r_i \pm u \times v$$

Theorem

If $\mathbb{K}$*-product,* $\mathbb{K}$*-addition and* $\mathbb{K}$*-subtraction are in* L*, then* SLP* $\subseteq$ CL

(using reversibility)





Ben-Or & Cleve

Definition (Algebraic Formulae over $\mathbb{K}$)

$$x_i \qquad c \in \mathbb{K} \qquad \begin{array}{ccc} & + & \\ f & & g \end{array} \qquad \begin{array}{ccc} & \times & \\ f & & g \end{array}$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$

Definition (Algebraic Formulae over $\mathbb{K}$)

$$x_i \qquad c \in \mathbb{K} \qquad \overset{+}{f \quad g} \qquad \overset{\times}{f \quad g}$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*

Definition (Algebraic Formulae over $\mathbb{K}$)

$$x_i \qquad c \in \mathbb{K} \qquad \overset{+}{f \quad g} \qquad \overset{\times}{f \quad g}$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*

Definition (Algebraic Formulae over $\mathbb{K}$)

$$x_i \qquad c \in \mathbb{K} \qquad \overset{+}{\underset{f \quad g}{\diagup \diagdown}} \qquad \overset{\times}{\underset{f \quad g}{\diagup \diagdown}}$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*

$$x_i \qquad c \in \mathbb{K}$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_i := r_i + u \times f$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_i := r_i + u \times f$

$r_i = \tau_i + u \times f$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_i := r_i + u \times f$
$r_i := r_i + u \times g$

$r_i = \tau_i + u \times f$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_i := r_i + u \times f$
$r_i := r_i + u \times g$

$r_i = \tau_i + u \times f$
$r_i = \tau_i + u \times (f + g)$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_j := r_j + u \times f$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_j := r_j + u \times f$

$r_j = \tau_j + u \times f$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_j := r_j + u \times f$
$r_i := r_i + r_j \times g$

$r_j = \tau_j + u \times f$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_j := r_j + u \times f$
$r_i := r_i + r_j \times g$

$r_j = \tau_j + u \times f$
$r_i = \tau_i + \tau_j g + u \times (f \times g)$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$r_j := r_j + u \times f$
$r_i := r_i + r_j \times g$
$r_j := r_j - u \times f$

$r_j = \tau_j + u \times f$
$r_i = \tau_i + \tau_j g + u \times (f \times g)$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$$r_j := r_j + u \times f$$
$$r_i := r_i + r_j \times g$$
$$r_j := r_j - u \times f$$

$$r_j = \tau_j + u \times f$$
$$r_i = \tau_i + \tau_j g + u \times (f \times g)$$
$$r_j = \tau_j$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$$r_j := r_j + u \times f$$
$$r_i := r_i + r_j \times g$$
$$r_j := r_j - u \times f$$
$$r_i := r_i - r_j \times g$$

$$r_j = \tau_j + u \times f$$
$$r_i = \tau_i + \tau_j g + u \times (f \times g)$$
$$r_j = \tau_j$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*



$$\begin{aligned} r_j &:= r_j + u \times f \\ r_i &:= r_i + r_j \times g \\ r_j &:= r_j - u \times f \\ r_i &:= r_i - r_j \times g \end{aligned}$$

$$\begin{aligned} r_j &= \tau_j + u \times f \\ r_i &= \tau_i + \tau_j g + u \times (f \times g) \\ r_j &= \tau_j \\ r_i &= \tau_i + u \times (f \times g) \end{aligned}$$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*

Observation

$size(P) \leq 4^{depth(f)}$

Ben-Or & Cleve

Theorem (Ben-Or & Cleve, STOC 1988)

For any formula f, there is a 3-register SLP^* *program computing:*

$$r_i := r_i \pm u \times f(\vec{x})$$

- $i \in \{1, 2, 3\}$
- *u is either a constant or a register r_k, $k \neq i$*
- *this program does not change r_j for $j \neq i$.*

Observation

$size(P) \leq 4^{depth(f)}$

Corollary

Iterated $\mathbb{K}$-product is in SLP^*

Ben-Or & Cleve

Corollary

Iterated $\mathbb{K}$*-product is in* SLP^*.

Ben-Or & Cleve

Corollary

Iterated $\mathbb{K}$*-product is in* SLP^**.*

Corollary

If $\mathbb{K}$-product, $\mathbb{K}$-addition and $\mathbb{K}$-subtraction are in L, then iterated $\mathbb{K}$-product is in CL.

Ben-Or & Cleve

Corollary

Iterated $\mathbb{K}$*-product is in* SLP^**.*

Corollary

If $\mathbb{K}$-product, $\mathbb{K}$-addition and $\mathbb{K}$-subtraction are in L, then iterated $\mathbb{K}$-product is in CL.

For $\mathbb{K}$ the appropriate ring (of n^2-bit integer matrices), we then get:

Corollary

$\#\mathsf{L} \subseteq \mathsf{CL}$.

More results

Theorem

$\mathsf{VP} \subseteq \mathsf{SLP}^*$ *(extend [BC88] to circuits with n-ary* $+$*, 2-ary* $\times$*)*

More results

Theorem
$\mathsf{VP} \subseteq \mathsf{SLP}^*$ *(extend [BC88] to circuits with n-ary* $+$*, 2-ary* $\times$*)*

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{SLP}^*$ *(extend the above with* $g(x) = x^n$ *gates over commutative rings)*

More results

Theorem
$\mathsf{VP} \subseteq \mathsf{SLP}^*$ *(extend [BC88] to circuits with n-ary* $+$*, 2-ary* $\times$*)*

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{SLP}^*$ *(extend the above with* $g(x) = x^n$ *gates over commutative rings)*

Theorem
$\mathsf{CL} \subseteq \mathsf{ZPP}$

More results

Theorem
$\mathsf{VP} \subseteq \mathsf{SLP}^*$ *(extend [BC88] to circuits with n-ary* $+$*, 2-ary* $\times$*)*

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{SLP}^*$ *(extend the above with* $g(x) = x^n$ *gates over commutative rings)*

Theorem
$\mathsf{CL} \subseteq \mathsf{ZPP}$ *(*CL *machines run in polynomial time on average over the choice of auxiliary tape content.)*

More results

Theorem
$\mathsf{VP} \subseteq \mathsf{SLP}^*$ *(extend [BC88] to circuits with n-ary* $+$*, 2-ary* $\times$*)*

Theorem
$\mathsf{TC}^1 \subseteq \mathsf{SLP}^*$ *(extend the above with* $g(x) = x^n$ *gates over commutative rings)*

Theorem
$\mathsf{CL} \subseteq \mathsf{ZPP}$ *(*CL *machines run in polynomial time on average over the choice of auxiliary tape content.)*

And a few more

A Conclusion

- **It is possible to use full memory in a non-trivial way!**

A Conclusion

- **It is possible to use full memory in a non-trivial way!**
- Much stronger results are needed for practical application!

A Conclusion

- **It is possible to use full memory in a non-trivial way!**
- Much stronger results are needed for practical application!
- We now believe: $\mathsf{CL} \not\subseteq \mathsf{L}$.

A Conclusion

- **It is possible to use full memory in a non-trivial way!**
- Much stronger results are needed for practical application!
- We now believe: $\mathsf{CL} \not\subseteq \mathsf{L}$.
- Now for some open problems...

General Questions

- Is there any potential practical use for these things?

General Questions

- Is there any potential practical use for these things?
- Is there any advantage in transparent computation (which makes no assumption on the initial memory contents) vs. usual reversible computation (which assumes that the initial memory is set to zero)?

General Questions

- Is there any potential practical use for these things?
- Is there any advantage in transparent computation (which makes no assumption on the initial memory contents) vs. usual reversible computation (which assumes that the initial memory is set to zero)?
- Is there some relationship between transparent/catalytic computation and homomorphic encryption?

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers).

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers). This follows from the following two papers:

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers). This follows from the following two papers:
  - The following paper shows balanced formulas of size s have tranparent programs of size $s^{1+\varepsilon}$: Richard Cleve. Towards optimal simulations of formulas by bounded-width programs. Computational Complexity, 1991, 1(1): 91–105.

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers). This follows from the following two papers:
  - The following paper shows balanced formulas of size s have tranparent programs of size $s^{1+\varepsilon}$: Richard Cleve. Towards optimal simulations of formulas by bounded-width programs. Computational Complexity, 1991, 1(1): 91–105.
  - The following paper shows how to balance any formula of size s into one of size $s^{1+\varepsilon}$: Maria Luisa Bonet and Samuel R Buss. Size-depth tradeoffs for Boolean formulae. Information Processing Letters, 1994, 49(3): 151–155

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers).

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers).
- Can we improve this for formulas? say to $s \log s \ldots$

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers).
- Can we improve this for formulas? say to $s \log s \ldots$
- Can we prove this $s^{1+\varepsilon}$ result for circuits?

Questions about transparent programs

- Can we do exponentiation over non-commutative rings?
- Can we do n-fold multiplication over commutative rings, while using the inductive hypothesis $O(1)$ times? (Or even $o(n)$?)
- How large must trnsp. programs be for circuits/formulas size s?
- We have seen s^2 for formulas. It is possible to get $s^{1+\varepsilon}$-long (skew) programs, instead (though they use more than 3 registers).
- Can we improve this for formulas? say to $s \log s \ldots$
- Can we prove this $s^{1+\varepsilon}$ result for circuits?
- Can we show any non-trivial upper-bound for formulas computed by transparent programs? The obvious time upper bound is P and space upper-bound is PSPACE; can any of these be improved? E.g. can we compute such transparent programs in fixed-polynomial space? Is $\mathrm{CL} \subseteq \mathrm{TC}_1$? NC_2?

Questions about CL

- Can we derandomize the ZPP upper-bound, i.e., is CL $\subseteq$ P?

Questions about CL

- Can we derandomize the ZPP upper-bound, i.e., is $\mathsf{CL} \subseteq \mathsf{P}$?
- Can we show $\mathsf{NC}^2 \subseteq \mathsf{CL}$?

Questions about CL

- Can we derandomize the ZPP upper-bound, i.e., is CL $\subseteq$ P?
- Can we show $\mathrm{NC}^2 \subseteq$ CL?
- Can we prove any fixed-polynomial upper-bound on the space required for computing CL, e.g.,
  - Is CL $\subseteq$ DSPACE(n^{10})?
  - Is CL $\subseteq$ DSPACE($\operatorname{polylog} n$)?

Questions about CL

- Can we derandomize the ZPP upper-bound, i.e., is CL $\subseteq$ P?
- Can we show $\mathsf{NC}^2 \subseteq$ CL?
- Can we prove any fixed-polynomial upper-bound on the space required for computing CL, e.g.,
  - Is CL $\subseteq$ DSPACE(n^{10})?
  - Is CL $\subseteq$ DSPACE($\operatorname{polylog} n$)?

THANK YOU

Computing on a Full Memory

Harry Buhrman	University of Amsterdam & CWI
Richard Cleve	University of Waterloo
Michal Koucký	Charles University, Prague
Bruno Loff	University of Porto & INESC-Tec
Florian Speelman	University of Copenhagen

